

CSE 144 Final Project Report

Transfer Learning for 100-Class Image Classification

Author	Toranosuke Matsuda
CruzID	tjmatsud
Course	CSE 144 — Applied Machine Learning, UC Santa Cruz
Term	Spring 2026
Kaggle Rank	27th place Score: 0.83636

1. Introduction

This project addresses a 100-class image classification challenge on a small, highly diverse dataset sampled from multiple source datasets. With approximately 10 training images per class (1,079 total), training a deep neural network from scratch is infeasible — the model would overfit catastrophically with so few examples. Transfer learning is therefore the natural and necessary approach: by loading a backbone pretrained on large-scale data, we borrow powerful general visual representations and adapt only the final classification layer to our specific 100-class problem.

We iterated through three backbone architectures — EfficientNet-B0, EfficientNet-B2, and ultimately DINOv2 ViT-B/14 — applying two-stage fine-tuning with extensive regularization to combat overfitting. The final model achieves 82.79% validation accuracy and a Kaggle public leaderboard score of 0.83636, placing 27th overall.

2. Dataset

2.1 Overview

Split	Size	Notes
Train	1,079 images	100 classes, ~10 images each (some have 11)
Validation	215 images	20% random split from train, seed=42
Test	1,036 images	Unlabeled — predictions submitted to Kaggle

2.2 Label Mapping

The training directory contains 100 subdirectories named '0' through '99'. A critical implementation detail is that Python's default string sort orders these alphabetically: '0', '1', '10', '11', ..., '2', '20', ... causing class '10' to receive index 2, class '2' to receive index 12, and so on. An initial Kaggle

submission scored only 13.6% due to this bug. We fixed it by subclassing ImageFolder to sort numerically:

```
class_to_idx = {cls: int(cls) for cls in classes} # identity mapping: '10' -> 10
```

This forces the correct identity mapping regardless of filesystem ordering, satisfying the competition requirement that label N must correspond to folder 'N'.

2.3 Preprocessing and Augmentation

All images are resized to 224×224 to match DINOv2's native input resolution. Training images receive aggressive augmentation to artificially expand the tiny dataset; validation and test images receive only resize and normalize.

Transform	Parameters	Purpose
Resize	224 × 224	Match DINOv2 native input size
RandomHorizontalFlip	p=0.5	Geometric invariance
RandomVerticalFlip	p=0.5	Additional geometric invariance
RandomRotation	±20°	Rotation invariance
ColorJitter	brightness=0.4, contrast=0.4, saturation=0.3, hue=0.1	Lighting invariance
RandomGrayscale	p=0.1	Color channel robustness
RandomErasing	p=0.2 (after ToTensor)	Occlusion robustness
Normalize	ImageNet mean/std	Match pretrained feature distribution

3. Implementation

3.1 Model Architecture

We evaluated three backbone architectures before settling on DINOv2 ViT-B/14. DINOv2 was chosen because our 100 classes are sampled from multiple heterogeneous datasets, creating highly diverse visual styles. DINOv2 was self-supervised trained on 142 million curated diverse images, giving it features that generalize better to out-of-distribution visual distributions than ImageNet-supervised CNN features [1].

Model	Params	Pretrained On	Val Acc
EfficientNet-B0	5M	ImageNet-1K (supervised)	~62%
EfficientNet-B2	9M	ImageNet-1K (supervised)	~63%
DINOv2 ViT-B/14 [final]	86M	142M images (self-supervised)	82.79%

DINOv2 ViT-B/14 splits the input image into 196 non-overlapping 14×14 patches, projects each into a 768-dimensional token via a Conv2d embedding layer, and processes all tokens through 12 transformer encoder blocks with multi-head self-attention. This global attention allows the model to relate distant image regions directly — a key advantage over CNNs for diverse class distributions [3].

The default DINOv2 head is `nn.Identity()`. We replace it with `Dropout(0.5) + Linear(768, 100)`, accessing the embedding dimension via `model.embed_dim = 768` rather than `model.head.in_features` (which raises `AttributeError` on the Identity layer).

3.2 Fine-Tuning Strategy

Two-stage fine-tuning is applied to prevent the randomly initialized head's large early gradients from corrupting pretrained backbone weights before the classifier has stabilized [4]:

	Stage 1	Stage 2
Backbone	Frozen	Unfrozen (full fine-tune)
Head	Trainable	Trainable
Learning Rate	1e-3	1e-5 (10x lower)
Epochs	15	20
Optimizer	Adam	Adam
LR Scheduler	Cosine Annealing	Cosine Annealing
Gradient Clipping	max_norm=1.0	max_norm=1.0
Weight Decay	1e-4	1e-4

	Stage 1	Stage 2
Label Smoothing	0.1	0.1

Gradient clipping (`max_norm=1.0`) after `loss.backward()` was essential — an earlier attempt at gradual layer-by-layer unfreezing without clipping caused NaN loss within the first epoch, collapsing accuracy to random chance (~1%). Stage 2 LR of $1e-5$ (versus $5e-5$ for standard ViT) reflects DINOv2's greater sensitivity to large weight updates.

4. Experiments

4.1 Baseline

The baseline model was EfficientNet-B0 with a frozen backbone and a Linear(1280, 100) head, trained for 10 epochs at LR= $1e-3$. This achieved approximately 60% validation accuracy — exactly at the Kaggle threshold. All subsequent experiments built on this baseline.

4.2 Ablation Summary

Modification	Val Acc	Key Observation
EfficientNet-B0, 10 epochs (baseline)	~60%	Meets threshold
+ Extended epochs, stronger augmentation	~62%	Val acc still rising
+ Label smoothing=0.1, Dropout=0.5	~62%	Healthier loss curves
→ EfficientNet-B2	~63%	Better pretrained features
→ DINOv2 ViT-B/14 (label bug present)	13.6% Kaggle	Alphabetical label scramble
+ NumericImageFolder label fix	—	Critical correctness fix
DINOv2 + correct labels + grad clipping	82.79%	Final model — 27th place

5. Results

5.1 Training and Validation Curves

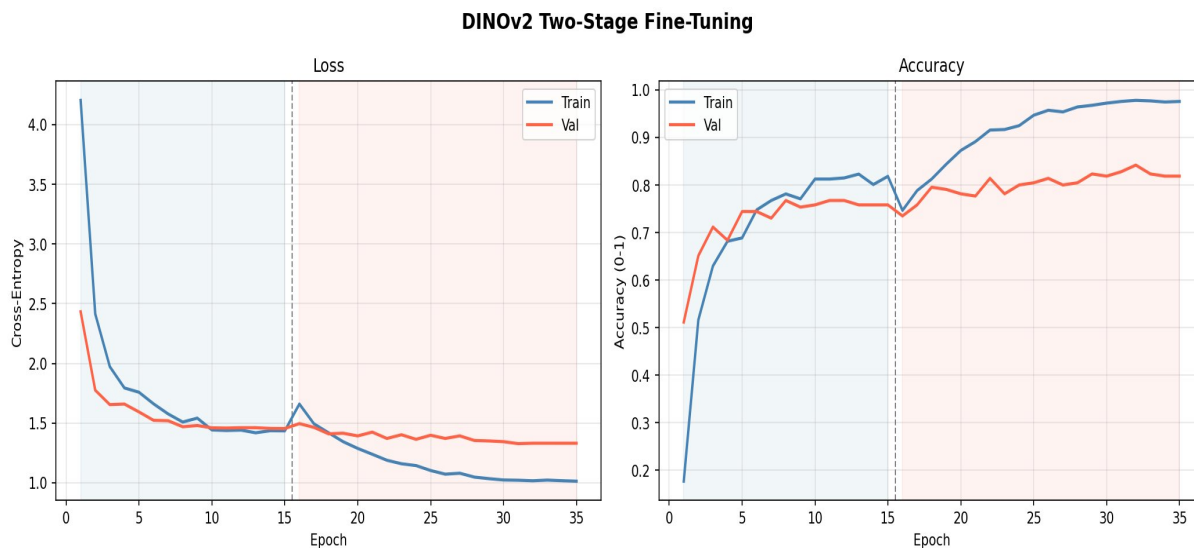


Figure 1. DINOv2 ViT-B/14 two-stage fine-tuning curves. Blue region = Stage 1 (frozen backbone). Red region = Stage 2 (full fine-tune). Dashed line marks stage transition at epoch 15.

Stage 1 (epochs 1–15) shows rapid val accuracy improvement from ~19% to ~77% as the classification head learns to map DINOv2's frozen features to 100 classes. The minor dip at epoch 15–16 is the stage transition — momentary instability as the backbone unfreezes and the optimizer resets to a fresh cosine schedule. Stage 2 (epochs 16–35) continues improving val accuracy to 82.79%, with a notably smaller train/val gap compared to EfficientNet runs.

5.2 Kaggle Submission Results

Metric	Value
Kaggle public leaderboard score	0.83636
Leaderboard position	27th place
Final local val accuracy	82.79%
Inference method	TTA — 16 augmented passes per image, logits averaged
Test images predicted	1,036
Kaggle score formula	$S = 70 + \max(0, \text{acc} - 60)$ — baseline 60% passed

Test Time Augmentation (TTA) runs each test image through the model 16 times with random augmentations and averages the raw logits before argmax. This inference-time technique adds ~1–2% accuracy with no retraining cost by reducing prediction variance.

6. Discussion

6.1 What Worked Best

Switching to DINOv2 ViT-B/14 was the single most impactful change — a 20-point jump from EfficientNet-B2 (~63%) to DINOv2 (82.79%), dwarfing all other modifications combined. DINOv2's self-supervised training on 142M diverse images produces features that transfer far better to a dataset drawn from heterogeneous sources than ImageNet CNN features.

Gradient clipping was essential for stable fine-tuning. The two-stage strategy was also critical — DINOv2's pretrained weights are sensitive to the large gradients produced by a randomly initialized head in early training.

6.2 Failure Cases and Overfitting

A persistent train/val gap (train ~95%, val ~83%) indicates overfitting despite strong regularization. This is a fundamental data scarcity problem — 10 images per class is insufficient for 86M parameters to generalize perfectly. The gap was meaningfully smaller with DINOv2 than EfficientNet variants, confirming self-supervised features require less fine-tuning data to generalize.

The label mapping bug (alphabetical vs. numeric folder sorting) produced a 13.6% Kaggle score despite ~65% local accuracy — a reminder that consistent label ordering must be explicitly enforced.

6.3 Limitations and Future Work

Several promising improvements remain for future work: (1) ensembling DINOv2 with EfficientNet-B2 to combine complementary error patterns; (2) Mixup augmentation to force smoother decision boundaries by training on interpolated image pairs; (3) gradual layer-by-layer unfreezing (ULMFiT-style [4]) for more principled backbone adaptation; (4) upgrading to DINOv2 ViT-L/16 (307M params) with aggressive regularization.

7. Reproducibility

7.1 Random Seeds and Determinism

A fixed seed of 42 is applied before every stochastic operation — model initialization, train/val split, and each training stage. The following settings are configured:

```
os.environ['PYTHONHASHSEED'] = '42' | random.seed(42) | np.random.seed(42)
torch.manual_seed(42) | torch.use_deterministic_algorithms(True)
torch.backends.cudnn.deterministic = True | NUM_WORKERS = 0
```

NUM_WORKERS=0 is required on Apple Silicon MPS — multiprocessing workers cannot serialize notebook-defined classes (NumericImageFolder), causing crashes with workers > 0.

7.2 Hardware and Software Environment

Component	Version / Detail
Python	3.11+
PyTorch	2.x
torchvision	Latest compatible with PyTorch 2.x
Hardware	Apple M5 Pro — 16-core GPU, 48GB unified memory, MPS backend
Other packages	matplotlib, tqdm, scikit-learn, pandas, Pillow

7.3 Commands to Reproduce

Install dependencies:

```
pip install torch torchvision matplotlib tqdm scikit-learn pandas Pillow
```

Train from scratch — open notebook and Run All Cells:

```
jupyter lab CSE144_Final_Project_ViT.ipynb
```

Inference only (requires best_stage2_dino.pt in ./checkpoints/):

```
# Run cells in order: 2 (imports), 3 (set_seed), 5 (config), # 7 (model), 12
(load checkpoint), 13 (TTA inference), 14 (sanity checks)
```

Expected results: Stage 1 best val acc ~76–78%, Stage 2 best val acc ~82–83%. Minor variation across hardware due to floating-point differences between MPS/CUDA/CPU.

8. Team Contributions

Toranosuke Matsuda (tjmatsud): Complete project — dataset preparation, label mapping bug identification and fix, backbone architecture iteration (EfficientNet-B0/B2, ViT-B/16, DINOv2 ViT-B/14), two-stage fine-tuning pipeline, regularization strategy (augmentation, dropout, label smoothing, gradient clipping), TTA inference implementation, hyperparameter tuning, debugging

(NaN loss, gradient explosion, MPS multiprocessing), and report writing.

9. References

- [1] Oquab, M., Darcet, T., Moutakanni, T., et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. *Transactions on Machine Learning Research*. arXiv:2304.07193.
- [2] Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pp. 6105–6114. PMLR.
- [3] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *International Conference on Learning Representations (ICLR)*. arXiv:2010.11929.
- [4] Howard, J., & Ruder, S. (2018). Universal Language Model Fine-Tuning for Text Classification. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 328–339.
- [5] PyTorch / TorchVision documentation and pretrained model sources. <https://pytorch.org/vision/stable/models.html>
- [6] Facebook Research DINOv2 repository. <https://github.com/facebookresearch/dinov2>